Freight Rate Prediction

Machine Learning Engineer assessment — validation approach, model and December forecast

1. Executive summary

- **Task.** Predict `posted_rate` for 12,000 loads dated Nov–Dec 2025, using 48,000 labelled loads from Jan–Oct 2025. The deliverable is a forecast 1–2 months past the last label.
- **Validation.** Forward-in-time splits only: model selection on train Jan–Aug / test Sep–Oct, confirmed by a 5-window rolling backtest and an unseen-city test. Random K-fold is reported only as a contrast: it rewards models that memorise each day's level.
- **Model.** Ridge regression on the daily market level and a linear time trend, plus LightGBM on the ridge residual. Target is log(rate per mile). The trend (0.64% per month on top of the market level) is the part tree models cannot carry past the last training date.
- **Results (clean-label MAPE).** Time holdout 2.05% vs 3.82% for the median baseline and 2.12% for LightGBM alone. Rolling backtest median 1.76% (wins 4 of 5 windows against LightGBM). Unseen cities 2.41%.
- **Data quality.** 677 corrupted rate labels (1.4%, off by 2–5× or 0.2–0.5×) removed from training only; sign-flipped, missing and censored weights repaired and flagged; missing market_index filled from the same day's market level.
- **Outputs.** `validation_predictions.csv` (12,000 rows, `load_id,predicted_rate`) and the December chart, both validated by the provided `score.py`. Spotter computes the final metric; no score on Nov–Dec is claimed here.

2. Data and exploratory findings

`train_test.csv`: 48,000 loads × 14 columns, 2025-01-01 to 2025-10-31. `validation.csv`: 12,000 loads, 2025-11-01 to 2025-12-31, same columns without the target. 64 cities in training; validation has 72, of which 8 never appear in training (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego). Equipment: Dry Van 57%, Reefer 25%, Flatbed 18%. No duplicate ids or duplicate rows.

- **Distance dominates.** log(rate) correlates 0.90 with distance; per-mile rates fall with distance. Modelling log(rate ÷ distance) gives a well-behaved target and multiplicative errors.
- **Market level, not the per-load value.** market_index varies by load, but its within-day spread is small (std ≈ 0.025). The daily mean correlates 0.70 with the daily rate level; a load's deviation from its day's mean adds almost nothing (0.08). The model uses the daily mean, built from the input column of both files (no labels involved).
- **Upward trend.** After the market level, rates drift up steadily: January sits below and Sep–Oct above what the market alone explains (figure 2).
- **Weight and calendar.** Per-mile rate rises ~7% from the lightest to the heaviest weight decile. Day of week moves rates ±1% (midweek high, weekend low). Pickup and delivery cities each shift rates by up to ±3%.
- **quote_signal** has no useful relationship with the rate (correlation 0.08 at load and day level). It is kept as a tree input but carries little weight.
- **No December history.** Training ends in October, so no December or holiday effect can be learned.

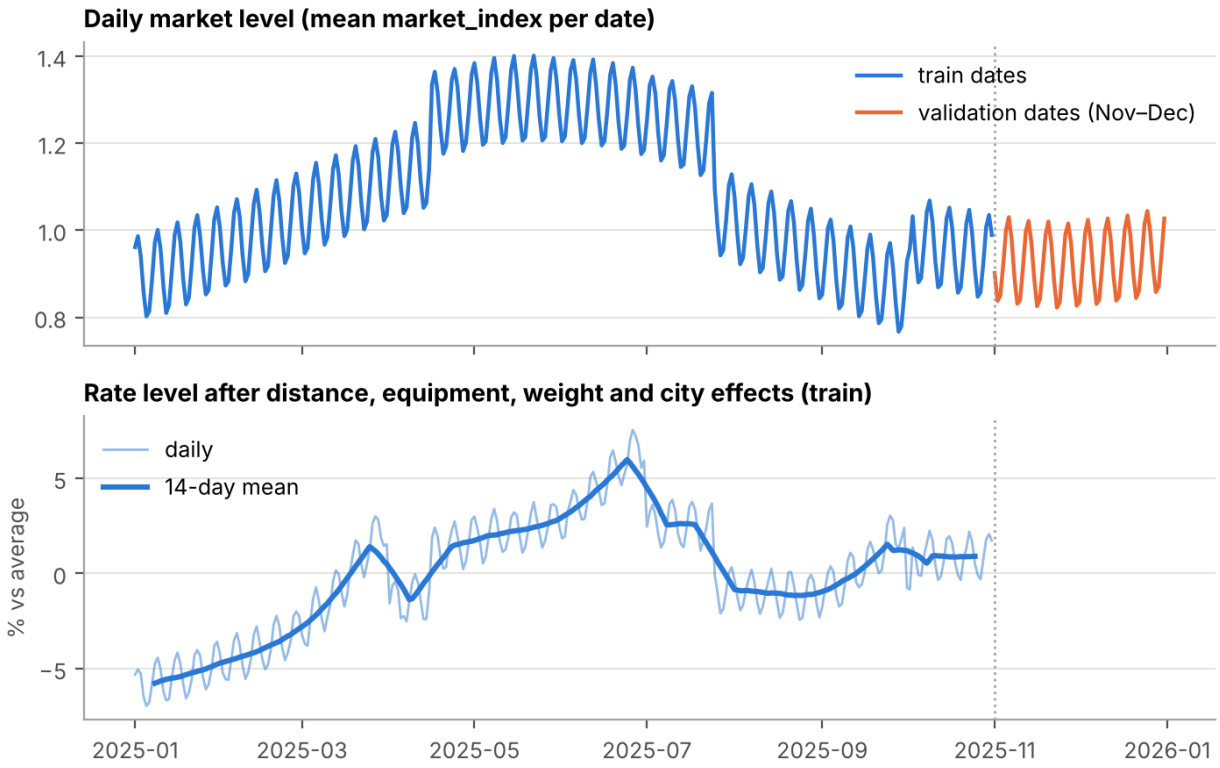


Figure 2. Top: daily market level; orange is the validation period. Bottom: the rate level left after distance, equipment, weight and city effects (training data).

3. Data-quality issues and treatment

Issue	Train	Validation	Treatment
missing weight	300	165	flag + equipment median (fit on training fold)
negative weight	292	145	sign error: absolute value + flag
weight at 47,500 cap	1204	301	censored: kept, flagged
missing market_index	374	249	flag + that date's market level
duplicate load_id	0	0	none found
corrupted rate label	677	–	dropped from training only (ratio outside 0.6–1.6× segment median)

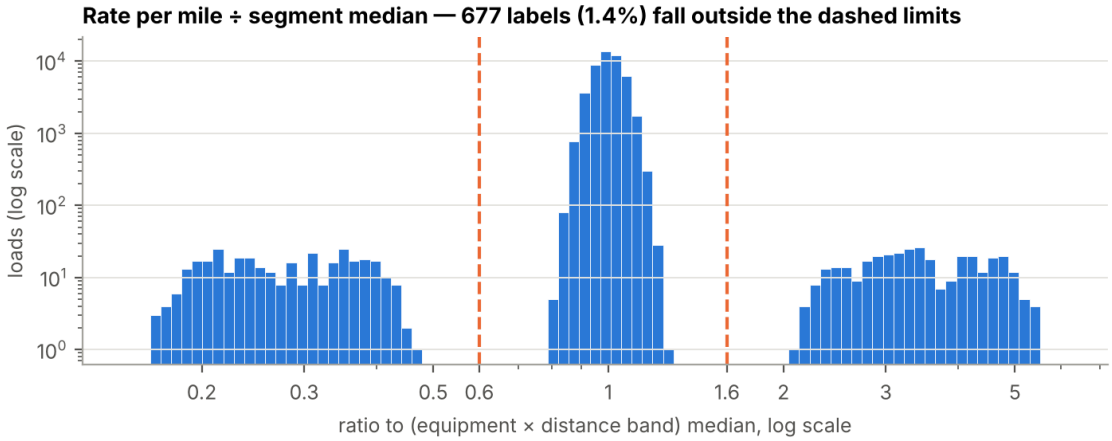


Figure 3. Rate per mile divided by its equipment × distance-band median. 98% of loads sit within 0.86–1.16×; the corrupted labels form separate clusters at 0.17–0.47× and 2.1–5.4×, so the 0.6× / 1.6× limits (dashed) separate them cleanly.

Repairs that need no learned statistics (sign fix, flags) apply to every row. Statistics are learned on the training fold only: imputation medians, outlier limits and city effects. The label filter only ever removes training rows, so every load that needs a prediction gets one.

4. Feature engineering and leakage prevention

- **Distance:** log distance, its square, 100/distance (fixed per-load charge on short hauls); circuity (road ÷ straight-line distance).
- **Load:** equipment one-hot; weight (repaired) with missing and censored-at-47,500 flags.
- **Market and time:** daily market level, the load's own market_index, days since 2025-01-01, day of week, day of month.
- **Geography:** pickup/delivery coordinates and smoothed pickup/delivery city effects. Training rows get 5-fold out-of-fold effects, so a row's label never feeds its own feature. Unseen cities get 0 (the average) and rely on coordinates.

Every feature is present in `validation.csv`; `posted_rate` is the only training-only column. Inside each validation fold the whole estimator is refit from scratch on the training part (label filter, imputation, city effects, model), so no test-fold label touches training.

5. Train / validation split

The submission is scored on Nov–Dec, after every training label. The validation must therefore test forecasting forward in time with the same horizon. A random split puts loads from the same days in both train and test, so the model sees each day's level before being tested on it. That is optimistic and, here, it picks the wrong model (section 6).

Split	Train	Test	n train / n test	Role
Time holdout	Jan 1 – Aug 31	Sep 1 – Oct 31	38,477 / 9,523	model selection
Rolling backtest	expanding from Jan 1	5 forward windows (2 months; last 1)	19,110–43,147 train	stability check
Unseen cities	Jan–Aug minus 8 random cities	Sep–Oct loads touching them	28,431 / 2,516	new-city check
Random 5-fold	random 80%	random 20%	38,400 / 9,600	contrast only

Random seed 42 is used for the K-fold split, the out-of-fold city effects, the unseen-city draw and the models. Scores are computed on all test rows and on test rows whose label passes the outlier filter (limits from the training part). Corrupted labels can't be predicted by any model, so clean-label MAPE is the selection metric. No official metric is computed by `score.py`, so MAPE is used (scale-free across $200 and $6,000 loads), with MAE, RMSE, R^2 and bias alongside.

Can the trend be identified? A linear trend can only be separated from the market level when the two are not moving together in the training window:

training window	corr(market, time)	market coef	trend %/month
Jan – Jun	0.86	0.083	1.36
Jan – Jul	0.73	0.141	0.69
Jan – Aug	0.35	0.16	0.52
Jan – Sep	0.03	0.141	0.66
Jan – Oct	-0.11	0.142	0.64

With Jan–Jun data the market and time are 0.86 correlated and the trend estimate doubles. From Jan–Aug onward it settles at 0.5–0.7% per month. The final model trains on Jan–Oct, where the correlation is −0.11: the best-identified window of all.

6. Models compared and selection

All candidates predict log(rate ÷ distance) and are converted back to dollars. (1) **Median baseline** per equipment × distance band. (2) **Ridge** on the linear features. (3) **Random forest** and (4) **LightGBM** on all features, including the day number. (5) **Ridge + LightGBM on residual**: ridge carries the market level and trend, which extrapolate; LightGBM fits the remaining non-linear structure without the day number, so it never has to extrapolate time.

Model	Holdout MAPE	Holdout MAPE (all rows)	MAE $	RMSE $	Bias %	Rolling mean / median	Unseen cities	Random 5-fold
Median baseline	3.82%	6.32%	82.8	112.9	-0.70	4.32% / 3.83%	3.82%	4.40%
Ridge	2.30%	4.80%	53.7	79.7	-1.77	3.23% / 1.99%	2.75%	1.82%
Random forest	2.27%	4.75%	57.0	84.4	-1.73	2.52% / 2.27%	2.45%	1.19%
LightGBM	2.12%	4.60%	53.8	80.6	-1.60	2.97% / 2.74%	2.33%	1.12%
Ridge + LightGBM (final)	**2.05%**	**4.55%**	**49.4**	**73.0**	**-1.70**	**3.00% / 1.76%**	**2.41%**	**1.32%**

Clean-label test rows unless marked. Holdout = train Jan–Aug, test Sep–Oct. MAE, RMSE and bias are holdout, clean labels.

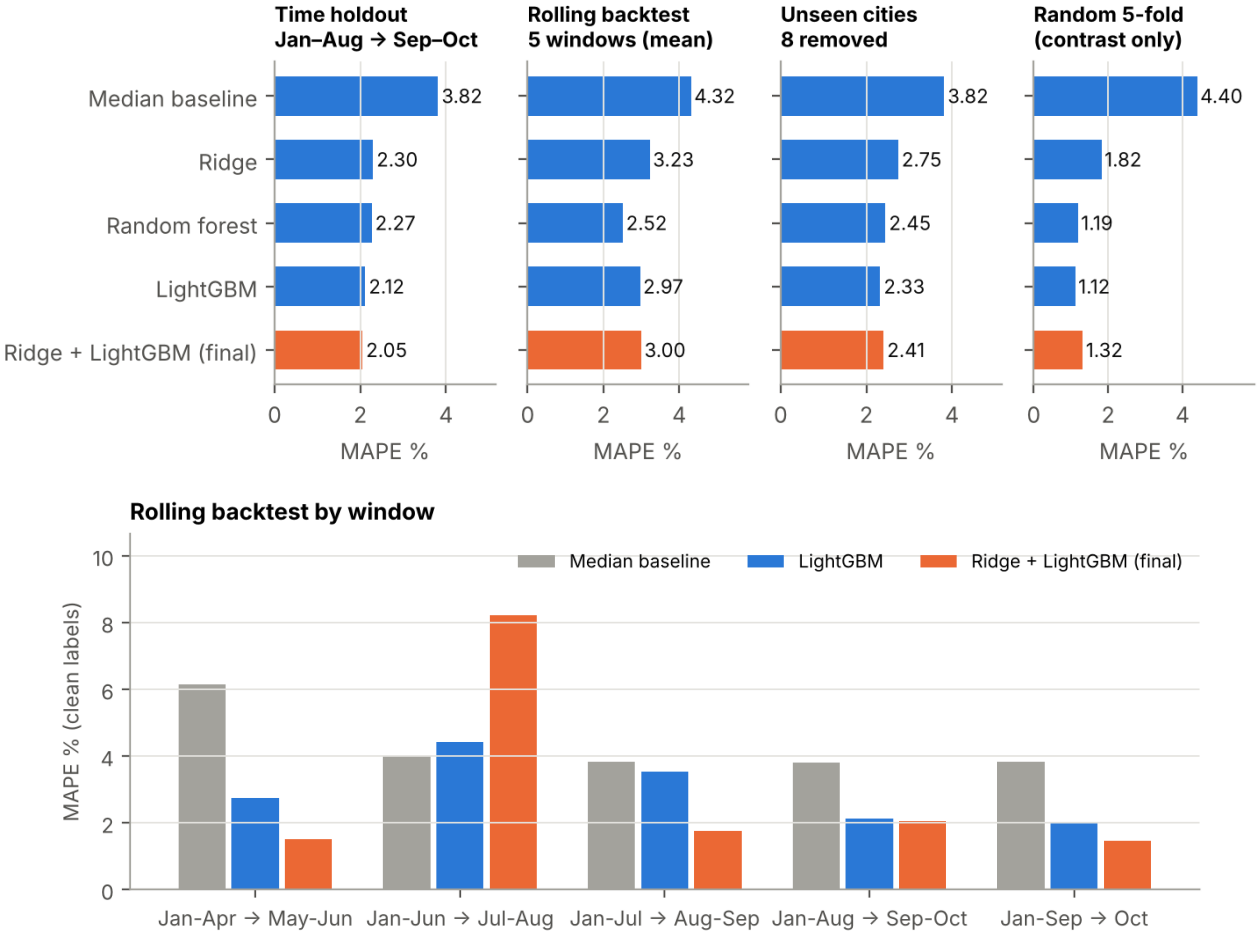


Figure 4. Clean-label MAPE by split, and per rolling window.

- Every learned model beats the median baseline on the average of every forward split. Per window there is one exception: in Jan–Jun → Jul–Aug the two trend-based models (ridge, final) do worse than the baseline.

- On the time holdout the final model has the lowest MAPE, MAE and RMSE.
- In the rolling backtest it wins 4 of 5 windows against LightGBM, by a wide margin (e.g. Jan–Jul → Aug–Sep: 1.76% vs 3.53%). It loses only Jan–Jun → Jul–Aug (8.2%), the one window where the trend can't be identified. That fold drags its mean to the level of the tree models; the median (1.76% vs 2.74% LightGBM, 2.27% random forest) shows the typical case.
- Random 5-fold ranks LightGBM and random forest first (≈1.1–1.2%) because they memorise each day's level. Those same models trail the final model on the forward holdout, which is why random K-fold is not used.
- Unseen cities cost about 0.35 points of MAPE.

Decision: Ridge + LightGBM on residual. It is the best model wherever the trend is identifiable. The final training window is the best-identified one, and it is the only candidate that carries the trend into Nov–Dec. A small grid (12 settings: ridge alpha, stage-2 leaves and trees) on the three identified rolling windows ranged 1.758–1.832% MAPE, with the defaults at 1.763%, so the defaults are kept.

7. Validation results of the final model and limitations

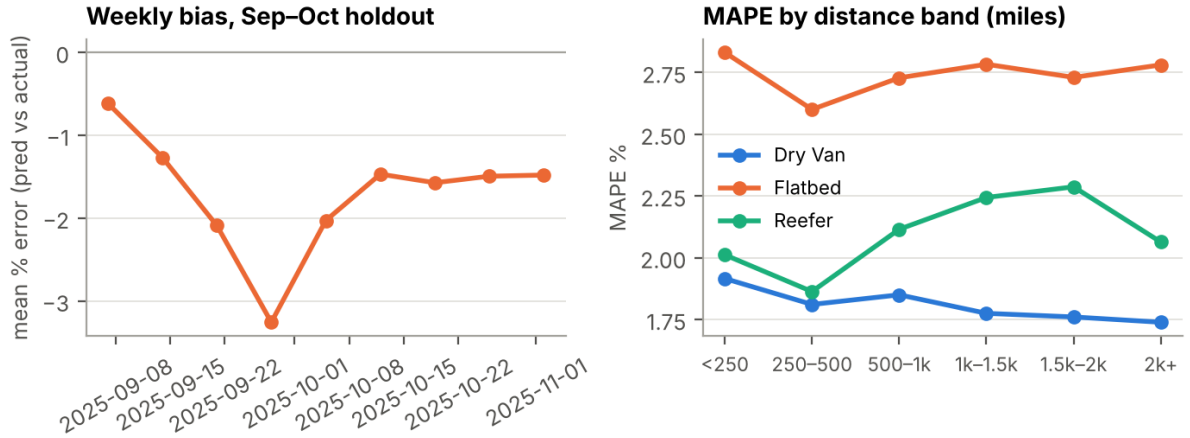
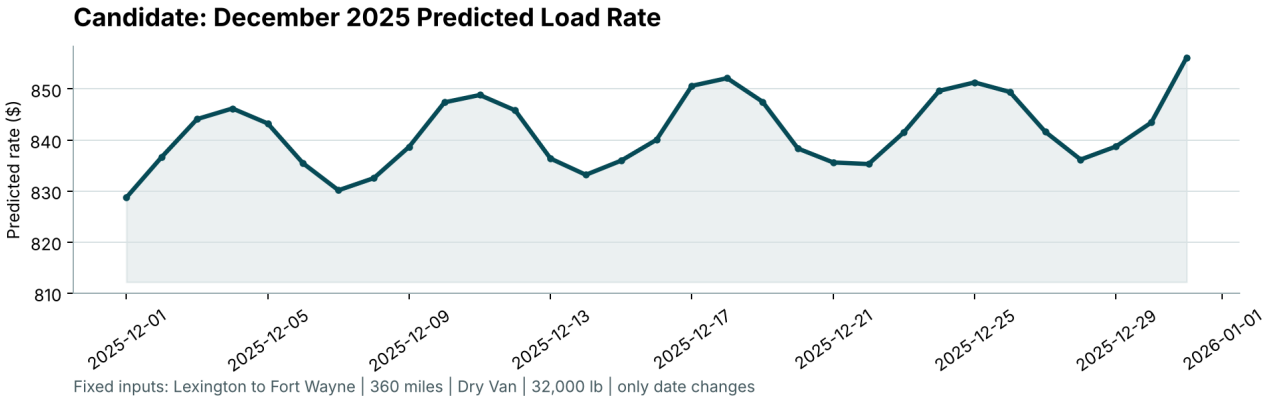


Figure 5. Final model on the Sep–Oct holdout (clean labels).

- Holdout (clean labels): MAPE 2.05%, MAE $49, RMSE $73, R^2 0.997. On all rows (including corrupted labels) MAPE is 4.55%; the gap comes from the 1.5% of test labels that are off by 2–5×.
- Errors are flat across distance bands (≈2.0–2.1% in each). By equipment: Dry Van 1.80%, Reefer 2.11%, Flatbed 2.73%.
- **Main limitation: level bias.** On the holdout the model under-predicts by 1.7% on average, with the same sign in every segment. Sep–Oct came in above what Jan–Aug implied. Most of the remaining error is this market-level uncertainty, which more training data narrows (the final fit has 2 more months and a better-identified trend) but can't remove.
- The linear trend is assumed to continue for two months. If the market reverses, Nov–Dec will be off by roughly the trend over the horizon (≈0.6–1.3%).
- No December or holiday data exists, so Christmas-week effects are not modelled.
- Corrupted labels in the hidden Nov–Dec set (if present at the training rate) will add error no model can remove; the all-rows MAPE above shows their size.
- December inputs not present in the file were filled: coordinates from the city table, market level from that date in `validation.csv`, quote_signal at its training median.

8. Fixed December prediction chart (from score.py)



Generated by the provided score.py from outputs/december_chart_predictions.csv, unmodified.

Fixed load: Lexington → Fort Wayne, 360 miles, Dry Van, 32,000 lb. Predictions range $828.73–$856.16. The shape is a weekly cycle (midweek peak, weekend dip) learned from the Jan–Oct day-of-week pattern, on a slight upward drift from the trend and the December market level. As a sanity check, the 21 clean Dry Van loads on this lane in Jan–Oct had a median rate of $807.90 at ~362 miles; the December predictions sit 2.6–6.0% above that, the direction the estimated trend implies.

`score.py` output: *Validated 12,000 final predictions. Validated 31 fixed December predictions.* The final predictions file has 12,000 rows, ids identical to and in the same order as the template, no missing or non-positive values (range $193.42–$6840.78, median $2062.98).

9. Reproducing the results

Put the four assessment CSVs in `data/` (train_test.csv, validation.csv, validation_predictions_template.csv, december_chart_inputs.csv), then:

```
pip install -r requirements.txt
python src/evaluate.py # all models × all splits -> artifacts/ (~7 min)
python src/tune.py # optional small grid (~4 min)
python src/train.py # final fit on Jan–Oct -> artifacts/final_model.joblib
python src/predict.py # -> outputs/validation_predictions.csv, december_chart_predictions.csv
python score.py --predictions outputs/validation_predictions.csv \
  --december-predictions outputs/december_chart_predictions.csv --output-dir outputs/scorer_results
python src/report.py # this PDF
```

Or run `notebooks/freight_rate_analysis.ipynb` top to bottom (Colab or local). Tested with Python 3.13.16, pandas 3.0.5, scikit-learn 1.9.1, LightGBM 4.7.0. The final model trains on 47,323 rows.